

ERD / API 명세서 수정사항 정리

1. 수정 목적

기존 API 명세서와 ERD는 기능 목록을 전체적으로 잘 포함하고 있었지만, 실제 백엔드와 프론트엔드 구현 단계에서 혼동될 수 있는 부분이 있었다.

이번 수정의 목적은 다음과 같다.

1. 백엔드와 프론트엔드가 같은 API 규칙을 기준으로 개발하기 위함
2. 데이터 저장 방식과 응답 형식을 통일하기 위함
3. 실제 확보한 음식/운동 데이터 기준에 맞게 불필요한 필드를 제거하기 위함
4. 로그인 여부에 따른 접근 정책을 명확히 하기 위함
5. 식단 기록, 루틴 기록처럼 하위 항목이 있는 API의 처리 방식을 명확히 하기 위함

2. API 공통 URL Prefix 추가

변경 전

```
/auth/login/  
/foods/  
/meals/  
/workouts/
```

변경 후

```
/api/v1/auth/login/  
/api/v1/foods/  
/api/v1/meals/  
/api/v1/workouts/
```

변경 이유

모든 API 주소 앞에 `/api/v1` 을 붙이기로 했다.

이유

1. 프론트엔드에서 API 기본 주소를 한 번에 관리할 수 있음
2. Django admin, 정적 파일, 일반 URL과 API URL을 구분하기 쉬움
3. 나중에 API 구조가 바뀌어도 v1, v2처럼 버전 관리 가능

프론트 사용 예시

```
const API_BASE_URL = 'http://127.0.0.1:8000/api/v1'
```

3. JWT 인증 방식 확정

변경 내용

로그인 방식은 JWT 기반 인증으로 확정했다.

로그인 성공 시 서버는 다음 두 토큰을 반환한다.

```
access token  
refresh token
```

access token

- API 요청 시 사용하는 토큰
- Authorization 헤더에 담아서 보냄
- 만료 시간이 짧음

refresh token

- access token을 재발급받기 위한 토큰
- access token보다 만료 시간이 김
- 로그아웃 시 서버에서 무효화 처리

인증이 필요한 API 요청 방식

```
Authorization: Bearer {access_token}
```

4. 로그아웃 방식 수정

변경 전

프론트에서 토큰만 삭제하는 방식도 고려 가능했음.

변경 후

서버에서 refresh token을 무효화하는 방식으로 확정했다.

```
POST /api/v1/auth/logout/  
Authorization: Bearer {access_token}
```

```
{  
  "refresh": "jwt_refresh_token"  
}
```

변경 이유

프론트에서 토큰만 삭제하면 사용자 화면에서는 로그아웃된 것처럼 보이지만, refresh token이 서버 기준으로는 여전히 유효할 수 있다.

따라서 완성도 있는 로그아웃 처리를 위해 서버에서 refresh token을 무효화한다.

5. API 응답 형식 통일

변경 전

API마다 응답 형식이 일부 달랐음.

예시:

```
{  
  "id": 1,  
  "name": "시래기 된장국"  
}
```

또는

```
{
  "success": true,
  "message": "조회 성공",
  "data": {}
}
```

처럼 섞일 수 있었음.

변경 후

모든 API 응답은 아래 형식으로 통일한다.

성공 응답

```
{
  "success": true,
  "message": "요청이 성공적으로 처리되었습니다.",
  "data": {}
}
```

실패 응답

```
{
  "success": false,
  "message": "요청 처리에 실패했습니다.",
  "errors": {
    "field": ["에러 메시지"]
  }
}
```

변경 이유

프론트엔드에서 항상 같은 방식으로 데이터를 꺼낼 수 있게 하기 위해서이다.

```
res.data.success
res.data.message
```

```
res.data.data
res.data.errors
```

이렇게 통일해서 사용할 수 있다.

6. 음식 영양성분 기준 확정

변경 내용

음식 데이터의 영양성분 기준을 100g 기준으로 확정했다.

```
calories
carbohydrate
protein
fat
```

위 값은 모두 음식 100g 기준 영양성분으로 본다.

예시

```
{
  "name": "시래기 된장국",
  "category": "국 및 탕류",
  "calories": 11.0,
  "carbohydrate": 1.6,
  "protein": 0.7,
  "fat": 0.26
}
```

위 데이터는 시래기 된장국 100g 기준 영양정보이다.

7. amount 단위 확정

변경 전

식단 기록에서 `amount` 가 무엇을 의미하는지 명확하지 않았음.

```
{
  "food_id": 1,
  "amount": 100
}
```

이 값이 100g인지, 1인분인지, 100ml인지 모호했음.

변경 후

`amount` 는 g 단위로 확정했다.

```
{
  "food_id": 1,
  "amount": 150
}
```

위 예시는 해당 음식을 150g 섭취했다는 의미이다.

계산식

```
섭취 칼로리 = 음식 칼로리 * amount / 100
섭취 탄수화물 = 음식 탄수화물 * amount / 100
섭취 단백질 = 음식 단백질 * amount / 100
섭취 지방 = 음식 지방 * amount / 100
```

예시

시래기 된장국 200g 섭취 시

```
칼로리 = 11.0 * 200 / 100 = 22.0 kcal
탄수화물 = 1.6 * 200 / 100 = 3.2g
단백질 = 0.7 * 200 / 100 = 1.4g
지방 = 0.26 * 200 / 100 = 0.52g
```

변경 이유

식단 기록, 저장 식단, AI 식단 추천에서 모두 같은 기준으로 영양성분을 계산하기 위해서이다.

8. AI 추천 결과 저장 방식 수정

변경 전 ERD

```
ai_recommendations.content text
```

AI 추천 결과를 단순 문자열로 저장하는 구조였다.

변경 후 ERD

```
content = models.JSONField()
```

변경 이유

AI 식단 추천과 AI 운동 추천 결과는 단순한 문장이 아니라 구조화된 데이터이다.

예를 들어 AI 식단 추천 결과는 다음과 같은 구조를 가진다.

```
{
  "total_calories": 2100,
  "total_carbohydrate": 240.0,
  "total_protein": 160.0,
  "total_fat": 55.0,
  "meals": [
    {
      "meal_type": "breakfast",
      "foods": [
        {
          "food_id": 1,
          "name": "시래기 된장국",
          "amount": 200,
          "calories": 22.0
        }
      ]
    }
  ]
}
```

이런 데이터를 TextField에 문자열로 저장하면 다시 꺼내서 사용하기 불편하다.
JSONField로 저장하면 다음 기능을 구현하기 쉽다.

1. 추천 결과 다시 조회
2. 추천 식단 저장
3. 추천 운동 루틴 저장
4. 프론트에서 구조화된 데이터로 바로 렌더링

9. 운동 데이터 필드 수정

변경 전

운동 상세 API와 화면설계에서 다음 필드가 포함되어 있었다.

난이도
MET
예상 소모 칼로리
소모 칼로리

변경 후

위 필드는 제거한다.

제거 대상

difficulty
MET
estimated_calories
calories_burned
total_burned_calories

변경 이유

현재 구축한 운동 데이터에는 다음 정보만 존재한다.

exercise_id
name
gif_url
body_parts


```
equipments
target_muscles
secondary_muscles
instructions
```

원본 운동 데이터에서 난이도, MET, 예상 소모 칼로리를 수집하지 않았기 때문에 정확한 값을 제공하기 어렵다.

부정확한 값을 임의로 넣는 것보다 실제 보유 데이터 기준으로 기능을 구성하는 것이 낫다고 판단했다.

최종 운동 상세 정보

운동 상세에서는 다음 정보만 제공한다.

```
운동명
운동 GIF
운동 부위
필요 장비
주요 자극 근육
보조 자극 근육
운동 수행 방법
```

10. 진행 현황 API 수정

변경 전

진행 현황에서 소모 칼로리도 포함될 수 있었음.

```
total_burned_calories
```

변경 후

소모 칼로리는 제외한다.

최종 진행 현황 데이터

```
체중 변화
총 섭취 칼로리
```

식단 점수
운동 횟수

변경 이유

운동 데이터에 MET 또는 소모 칼로리 계산 기준이 없기 때문에 정확한 소모 칼로리 산출이 어렵다.

따라서 진행 현황은 현재 정확하게 계산 가능한 데이터 중심으로 구성한다.

11. 음식 목록 조회 인증 정책 수정

변경 전

음식 목록 조회가 인증 필요로 되어 있었다.

GET /foods/ 인증 필요

변경 후

음식 목록 조회는 인증 선택으로 변경한다.

GET /api/v1/foods/ 인증 선택

정책

사용자 상태	조회 가능 데이터
비로그인	기본 음식 DB
로그인	기본 음식 DB + 내가 추가한 음식
로그인 + mine=true	내가 추가한 음식만

변경 이유

기본 음식 데이터는 서비스 공용 데이터이므로 로그인하지 않아도 조회 가능하게 하는 것이 자연스럽다.

단, 사용자가 직접 추가한 음식은 개인 데이터이므로 로그인한 사용자만 조회할 수 있다.

12. 운동 목록 조회 인증 정책 수정

변경 전

운동 목록 조회가 인증 필요로 되어 있었다.

```
GET /workouts/ 인증 필요
```

변경 후

운동 목록 조회는 인증 선택으로 변경한다.

```
GET /api/v1/workouts/ 인증 선택
```

정책

사용자 상태	조회 가능 데이터
비로그인	기본 운동 DB
로그인	기본 운동 DB + 내가 추가한 운동
로그인 + mine=true	내가 추가한 운동만

변경 이유

기본 운동 데이터는 서비스 공용 데이터이므로 로그인하지 않아도 조회 가능하게 하는 것이 자연스럽다.

단, 사용자가 직접 추가한 운동은 개인 데이터이므로 로그인한 사용자만 조회할 수 있다.

13. 식단 항목 수정 방식 변경

변경 전

식단 기록 수정 API에서 식단 자체와 음식 항목을 한 번에 수정할 수 있는 구조를 고려했다.

```
PATCH /api/v1/meals/{meal_id}/
```

변경 후

식단 자체 수정과 식단 항목 수정을 분리한다.

식단 자체 수정

```
PATCH /api/v1/meals/{meal_id}/
```

수정 대상

```
meal_type  
intake_date
```

식단 항목 추가

```
POST /api/v1/meals/{meal_id}/items/
```

식단 항목 수정

```
PATCH /api/v1/meal-items/{meal_item_id}/
```

식단 항목 삭제

```
DELETE /api/v1/meal-items/{meal_item_id}/
```

변경 이유

식단 기록은 `meals` 와 `meal_items` 가 부모-자식 관계로 나뉘어 있다.

음식 항목을 별도 API로 분리하면 다음 장점이 있다.

1. 특정 음식만 추가/수정/삭제하기 쉬움
2. 프론트에서 식단 편집 UI를 만들기 쉬움
3. 서버에서 변경 후 총 칼로리와 탄단지를 다시 계산하기 명확함
4. `meals` API가 너무 복잡해지는 것을 방지할 수 있음

14. 운동 루틴 항목 수정 방식 추가

변경 전

운동 루틴 수정에서 루틴 정보와 루틴 항목 수정 방식이 명확하지 않았음.

변경 후

루틴 자체 수정과 루틴 항목 수정을 분리한다.

루틴 자체 수정

```
PATCH /api/v1/workout-routines/{routine_id}/
```

수정 대상

```
title
description
goal
```

루틴 항목 추가

```
POST /api/v1/workout-routines/{routine_id}/items/
```

루틴 항목 수정

```
PATCH /api/v1/routine-items/{routine_item_id}/
```

루틴 항목 삭제

```
DELETE /api/v1/routine-items/{routine_item_id}/
```

변경 이유

운동 루틴도 `workout_routines` 와 `routine_items` 가 부모-자식 관계로 나뉘어 있다.

루틴 항목을 분리하면 운동 순서, 세트 수, 반복 횟수, 운동 시간을 개별적으로 수정하기 쉽다.

15. 커뮤니티 조회 인증 정책 정리

게시글 목록 / 상세

```
GET /api/v1/posts/  
GET /api/v1/posts/{post_id}/
```

인증 선택으로 둔다.

이유

커뮤니티 글은 비로그인 사용자도 조회할 수 있도록 한다.

단, 로그인 사용자의 경우 게시글 상세 조회 시 `is_liked` 값을 함께 받을 수 있다.

작성 / 수정 / 삭제 / 댓글 / 좋아요

아래 기능은 인증 필요로 유지한다.

```
POST /api/v1/posts/  
PATCH /api/v1/posts/{post_id}/  
DELETE /api/v1/posts/{post_id}/  
POST /api/v1/posts/{post_id}/comments/  
PATCH /api/v1/comments/{comment_id}/  
DELETE /api/v1/comments/{comment_id}/  
POST /api/v1/posts/{post_id}/like/
```

이유

데이터를 생성하거나 수정하는 기능은 사용자 식별이 필요하기 때문이다.

16. 사용자 공개 프로필 조회 인증 정책 정리

변경 후

```
GET /api/v1/users/{user_id}/public-profile/
```

인증 선택으로 둔다.

이유

커뮤니티에서 작성자 정보를 조회하는 목적이므로 기본 공개 프로필은 비로그인 사용자도 볼 수 있게 한다.

단, 민감한 정보는 제외하고 공개 가능한 정보만 반환한다.

17. 오타 및 명칭 수정

수정한 내용

DELTE → DELETE

운동 루틴 상세 조회 → 운동 루틴 상세 조회

운동기록 / 운동 기록 → 운동 기록으로 통일

게시글좋아요 → 게시글 좋아요

댓글작성 → 댓글 작성

음식목록 검색 → 음식 목록/검색

변경 이유

API 명세서는 백엔드, 프론트엔드, 발표 자료의 기준 문서이기 때문에 오타가 있으면 구현 시 혼동이 생길 수 있다.

18. ERD 수정사항 요약

18.1 ai_recommendations 테이블

변경 전

content text

변경 후

```
content json
```

이유

AI 추천 결과가 식단 목록, 운동 루틴 목록처럼 구조화된 데이터이기 때문이다.

18.2 workouts 테이블

제거 또는 제외 대상

```
difficulty  
MET  
estimated_calories  
calories_burned
```

이유

현재 운동 데이터에 해당 정보가 없기 때문이다.

최종 사용 필드

```
exercise_id  
name  
gif_url  
body_parts  
equipments  
target_muscles  
secondary_muscles  
instructions  
user_id  
created_at  
updated_at
```

18.3 progress_records 테이블

제거 또는 제외 대상

```
total_burned_calories
```


이유

운동별 소모 칼로리 계산 기준이 없기 때문에 정확한 값을 계산하기 어렵다.

최종 진행 현황 기준

```
weight
total_intake_calories
meal_score
workout_count
```

18.4 foods 테이블

유지

```
name
category
calories
carbohydrate
protein
fat
user_id
created_at
updated_at
```

보충 설명

기본 음식 fixture 데이터는 `user_id = null` 로 저장한다.

사용자가 직접 추가한 음식은 `user_id` 에 사용자 ID를 저장한다.

18.5 workouts 테이블의 user_id 기준

기본 운동 fixture 데이터는 `user_id = null` 로 저장한다.

사용자가 직접 추가한 운동은 `user_id` 에 사용자 ID를 저장한다.

19. API 명세서 수정사항 요약

19.1 추가된 공통 규칙

/api/v1 prefix
JWT 인증 방식
Authorization: Bearer {access_token}
공통 성공 응답 형식
공통 실패 응답 형식
공통 HTTP Status Code
음식 영양성분 100g 기준
amount g 단위
Enum 값 정리

19.2 추가된 API

```
POST /api/v1/meals/{meal_id}/items/  
PATCH /api/v1/meal-items/{meal_item_id}/  
DELETE /api/v1/meal-items/{meal_item_id}/  
  
POST /api/v1/workout-routines/{routine_id}/items/  
PATCH /api/v1/routine-items/{routine_item_id}/  
DELETE /api/v1/routine-items/{routine_item_id}/
```

19.3 수정된 API

모든 URL에 /api/v1 prefix 추가

```
GET /foods/  
→ GET /api/v1/foods/  
  
GET /workouts/  
→ GET /api/v1/workouts/  
  
GET /posts/  
→ GET /api/v1/posts/
```

19.4 인증 정책 변경

API	변경 내용
음식 목록/상세	인증 필요 → 인증 선택
운동 목록/상세	인증 필요 → 인증 선택
게시글 목록/상세	인증 선택 유지

API	변경 내용
사용자 공개 프로필	인증 선택
사용자 추가 음식/운동	인증 필요
식단 기록/루틴/운동 기록	인증 필요
AI 기능	인증 필요
댓글/좋아요	인증 필요

20. 최종 개발 기준

이번 수정 이후 개발 기준은 다음과 같다.

1. 모든 API는 /api/v1로 시작한다.
2. 로그인 필요한 API는 Authorization: Bearer access_token을 사용한다.
3. 로그아웃은 refresh token을 서버에서 무효화한다.
4. 모든 응답은 success, message, data, errors 형식으로 통일한다.
5. 음식 영양성분은 100g 기준이다.
6. 식단 기록의 amount는 g 단위이다.
7. 음식/운동 기본 DB는 비로그인 사용자도 조회 가능하다.
8. 사용자 추가 데이터와 개인 기록 데이터는 로그인 사용자만 접근 가능하다.
9. AI 추천 결과는 JSONField로 저장한다.
10. 운동 난이도, MET, 예상 소모 칼로리는 사용하지 않는다.
11. 식단 항목과 루틴 항목은 별도 API로 추가/수정/삭제한다.
12. 진행 현황에서는 소모 칼로리를 제외한다.

21. 확인 사항

백엔드

1. urls.py에서 /api/v1 prefix 적용
2. JWT 로그인/로그아웃 처리 확인
3. ai_recommendations.content를 JSONField로 변경
4. 음식 amount 계산 로직 서버에서 처리
5. meal_items 추가/수정/삭제 API 구현
6. routine_items 추가/수정/삭제 API 구현
7. 운동 난이도/MET/소모칼로리 관련 필드 제거
8. 공통 응답 형식 통일

프론트엔드

1. axios baseURL을 /api/v1 기준으로 설정
2. 로그인 후 access token을 Authorization 헤더에 포함
3. 로그아웃 시 서버 logout API 호출 후 로컬 토큰 삭제
4. 음식/운동 목록은 비로그인 상태에서도 조회 가능하게 처리
5. 식단 기록 amount는 g 단위로 입력받기
6. API 응답은 res.data.data 기준으로 사용
7. success=false인 경우 message 또는 errors 표시

AI

1. AI 식단 추천 결과를 JSON 구조로 반환
2. AI 운동 추천 결과를 JSON 구조로 반환
3. ai_recommendations.content에 JSON 형태로 저장
4. 추천 식단 저장 시 content 내부 foods 데이터를 saved_meals로 변환
5. 추천 운동 저장 시 content 내부 items 데이터를 workout_routines로 변환

중요

/api/v1 prefix
JWT 로그아웃
공통 응답 형식
amount g 단위
AI 추천 결과 JSONField 저장
운동 소모 칼로리 관련 필드 제거
식단/루틴 항목 별도 API 분리